

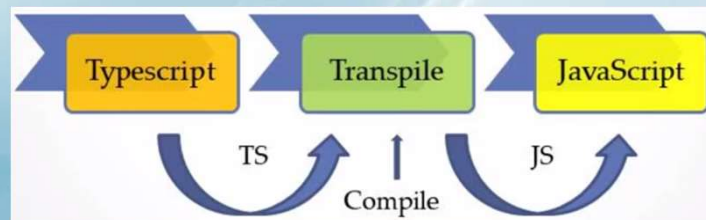


Type Script at a Glance



- Pretty obvious we're going to use Type Script as a coding language in Angular application development.
- What is Typescript? Typescript, really is just a superset of JavaScript.
- It offers more features than vanilla JavaScript, like classes, interfaces and, very important, which gives it the name, types; strong typing.
- However, Typescript doesn't run in the browser, so it is compiled to JavaScript in the end. This compilation is handled by the CLI.
- The process is actually known as TRANSPILE not COMPILE.
- A lot of the features really only exist in Typescript and Angular is meant to be used together with Typescript.
- In a fact you don't have to learn Typescript you should be able to pick it up along the way.

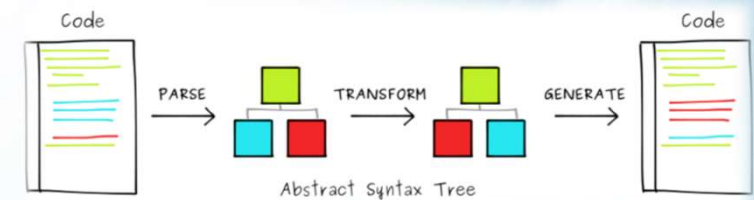
Micro Soft
Product





Transpiler

- Transpilers, or source-to-source compilers, are tools that read source code written in one programming language, and produce the equivalent code in another language that has a similar level of abstraction.
- A good example of transpiler is the Typescript transpiler, which converts Typescript code to JavaScript. Babel transpiler can also be used for ES6 JS code to ES5 JS code.
- A transpiler is a compiler like program, which convert ES6 JavaScript code to ES5 JavaScript code so that it can run in browsers.
- When the transpiler sees an expression that uses a language feature that needs to be translated, it produces a logically equivalent expression.
- The produced expressions can be very similar, or very different from the source expression.





Get TypeScript on Your System

- Let's install and use TypeScript on your PC.
- Before TypeScript you need NodeJS installed on the system. You can see the version installed on your system using command:

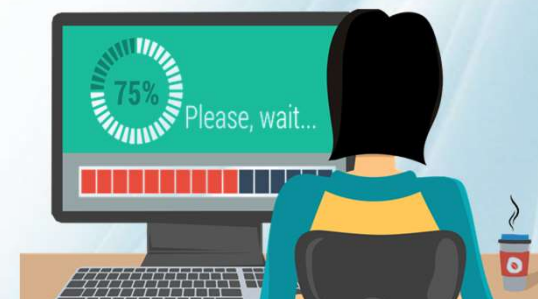
> node -v

- If it is not present, than first get it from <https://nodejs.org>
- There are two main ways to get the TypeScript available for your project:
 - Via npm (the Node.js package manager)
 - By installing TypeScript's Visual Studio plugins
- For npm users:

> npm install -g typescript

- You can check version installed on your system by command

> tsc -v



```
D:\2020\Angular\TS-Samples>node -v
v14.8.0
D:\2020\Angular\TS-Samples>tsc -v
Version 3.9.7
D:\2020\Angular\TS-Samples>_
```



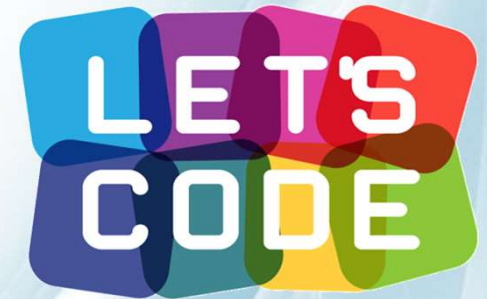
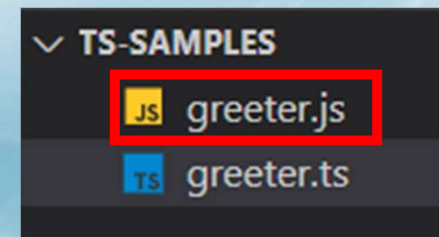
TypeScript in the Meantime

- Let's get started by building a simple web application with TypeScript.
- Building your first TypeScript file
- In your editor, type the following JavaScript code in greeter.ts.
- Compiling your code
- We used a .ts extension, but this code is just JavaScript.
- At the command line, run the TypeScript compiler:

> tsc greeter.ts

- The result will be a file greeter.js

```
1 function greeter(person) {  
2   return "Hello, " + person;  
3 }  
4  
5 let user = "Ranjan Bhatnagar";  
6  
7 document.body.textContent = greeter(user);
```



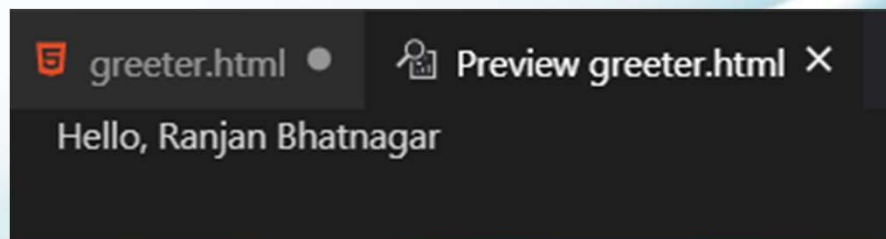


Running your TypeScript Web App



- The greeter.js contains the same JavaScript that you fed in greeter.ts.
- We're up and running using TypeScript in our JavaScript app!
- Now type the following in greeter.html:
- Run using any browser.
- In Visual Studio Code you can use right click on the file name and use Open Preview (Ctrl+Shift+V).

```
greeter.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>TypeScript Greeter</title>
5    </head>
6    <body>
7      <script src="greeter.js"></script>
8    </body>
9  </html>
```





Use Visual Studio Code



- Let's do Hello World! program of TypeScript in Visual Studio Code.
- Create one folder (for e.g. TYPESCRIPT) and open it in Visual Studio Code.
- Now create one TypeScript file named main.ts and write following →
- Save it and open Terminal Window(Ctrl+`)
- Compile/Transpile using cmdlet

... \TYPESCRIPT> tsc main.ts

- You can see main.js file in folder as result of it →
- Open main.js, you will see similar code except let is changed to var.
- To run go to the terminal and use cmdlet

... \TYPESCRIPT> node main.ts

- You can view output of log in Terminal →

```
main.ts > ...  
1 let message='Hello World!';  
2 console.log(message);
```

```
✓ TYPESCRIPT  
  JS main.js  
  TS main.ts
```

```
PS D:\2020\Angular\TYPESCRIPT> tsc main.ts  
PS D:\2020\Angular\TYPESCRIPT> node main.js  
Hello World!  
PS D:\2020\Angular\TYPESCRIPT> |
```





Why an Error?

- Notice that we have an error in let statement → `message`.
- Hover it and see message.
- This happens because the file is treated as script rather than a module.
- Module has its own scope whereas script shares the global scope.
- In this project `main.ts` and `main.js`, two files, share global scope hence error flashes. To solve the error we need to add an export statement at the top that exports nothing.
- After adding export/import statement, TypeScript treats this file as module instead of script.
- Changes reflect in `main.js` file also.



```
main.ts > ...
1 let message='Hello World!';
2 const let message: string

Cannot redeclare block-scoped variable 'message'. ts(2451)
main.js(1, 5): 'message' was also declared here.
Peek Problem (Alt+F8) No quick fixes available
```

```
main.ts > ...
1 export {}
2 let message='Hello World!';
3 console.log(message);
```

```
main.js > ...
1 "use strict";
2 exports.__esModule = true;
3 var message = 'Hello World!';
4 console.log(message);
```





Watch Changes

- If we want we can add an automatic compilation features with the project.
- It will automatically compiles .ts file as we make changes and save it.
- To do so add --watch with the cmdlet as:

> tsc main.ts --watch

- Make changes in main.ts and save it. Same time look at activities in the terminal window and see the main.js file which reflects the changes.

```
PS D:\2020\Angular\TYPESCRIPT> tsc main.ts --watch
[4:08:20 PM] Starting compilation in watch mode...

[4:08:23 PM] Found 0 errors. Watching for file changes.

[4:09:20 PM] File change detected. Starting incremental compilation...

[4:09:20 PM] Found 0 errors. Watching for file changes.
```

```
PS D:\2020\Angular\TYPESCRIPT> node main.js
Hello India!
PS D:\2020\Angular\TYPESCRIPT> █
```

- As watch mode is on, to run node command, open a new terminal window (Ctrl+Shift+`) and use cmdlet:

>node main.js

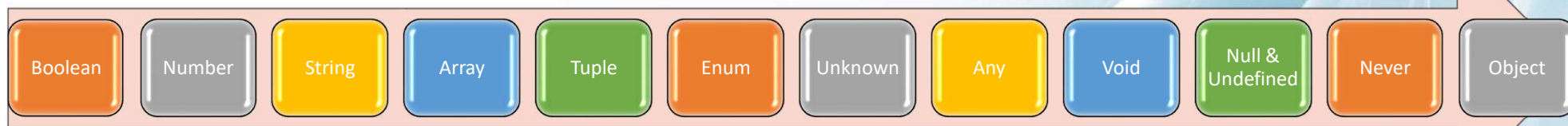




Basic Types & Variable Declaration



- For programs to be useful, we need to be able to work with some of the simplest units of data: numbers, strings, structures and bool values.
- There are basic data types which are listed below:



- Declaring a variable in JavaScript has always traditionally been done with the var keyword for example:

var a = 10;

let a = 10;

const a = 10;

- let and const are two relatively new concepts for variable declarations in JavaScript. let is similar to var in some respects, whereas const prevents re-assignment to a variable.





Few Examples

- Boolean → `let isDone: boolean = false;`
- Number → `let decimal: number = 6;`
`let hex: number = 0xf00d;`
`let binary: number = 0b1010;`
`let octal: number = 0o744;`
`let big: bigint = 100n;`
- String → `let color: string = "blue";`
`color = "red";`
`let fullName: string = `Bob Bobbington`;`
`let age: number = 37;`

```
let sentence: string =
  "Hello, my name is " +
  fullName +
  ".\n\n" +
  "I'll be " +
  (age + 1) +
  " years old next month.";
```

This is
equivalent to
declaring
sentence like

```
let sentence1: string = `Hello, my name is ${fullName}.
I'll be ${age + 1} years old next month.`;
```

Aka Template Strings

Presented by : *Ranjan Bhatnagar*
 bhatnagar_ranjan@outlook.com

Friday, January 29, 2021



ANGULAR





Points to Focus on

- The most basic datatype is the simple true/false value, which JavaScript and TypeScript call a boolean value.
- As in JavaScript, all numbers in TypeScript are either floating point values or BigIntegers.
- Just like JavaScript, TypeScript also uses double quotes (") or single quotes (') to surround string data.
- You can also use template strings, which can span multiple lines and have embedded expressions. These strings are surrounded by the backtick/backquote (`) character, and embedded expressions are of the form `${ expr }`.
- Array types can be written in one of two ways as:
 - Datatype followed by `[ ]` as `→ let list: number[] = [1, 2, 3];`
 - A generic array type `Array<elemType>` `→ let list: Array<number> = [1, 2, 3];`





More on Datatypes

- **Tuple** types allow you to express an array with a fixed number of elements whose types are known, but need not be the same. For example, you may want to represent a value as a pair of a string and a number.
- **Unknown** type allow you to describe the type of variables that we do not know when we are writing an application.
- **Any** In some situations, not all type information is available or it's declaration would take an inappropriate amount of effort. These may occur for values from code that has been written without TypeScript or a 3rd party library. In these cases, we might want to opt-out of type checking. To do so, we label these values with the any type.

- Tuple →

```
// Declare a tuple type
let x: [string, number];
// Initialize it
x = ["hello", 10]; // OK
// Initialize it incorrectly
x = [10, "hello"]; // Error
```
- Unknown →

```
let notSure: unknown = 4;
notSure = "maybe a string instead";
// OK, definitely a boolean
notSure = false;
```
- Any →

```
declare function getValue(key: string): any;
// OK, return value of 'getValue' is not checked
const str: string = getValue("myString");
```





Few More Datatypes

- **Void** → void is a little like the opposite of any: the absence of having any type at all. You may commonly see this as the return type of functions that do not return a value.
- Declaring variables of type void is not useful because you can only assign null (only if --strictNullChecks is not specified, see next section) or undefined to them
- **Null and Undefined** → In TypeScript, both undefined and null actually have their own types named undefined and null respectively. Much like void, they're not extremely useful on their own.
- **Never** → The never type represents the type of values that never occur.

- Void →

```
function warnUser(): void {  
    console.log("This is my warning message");  
}
```



```
let unusable: void = undefined;  
// OK if `--strictNullChecks` is not given  
unusable = null;
```
- Null and Undefined →

```
// Not much else we can  
// assign to these variables!  
let u: undefined = undefined;  
let n: null = null;
```
- Never →

```
// Function returning never must not  
// have a reachable end point  
function error(message: string): never {  
    throw new Error(message);  
}  
  
// Inferred return type is never  
function fail() {  
    return error("Something failed");  
}  
  
// Function returning never must not  
// have a reachable end point  
function infiniteLoop(): never {  
    while (true) {}  
}
```





Type Assertions

- Sometimes you'll end up in a situation where you'll know more about a value than TypeScript does.
- Usually this will happen when you know the type of some entity could be more specific than its current type.
- Type assertions are a way to tell the compiler "trust me, I know what I'm doing."
- A type assertion is like a type cast in other languages, but performs no special checking or restructuring of data.
- It has no runtime impact, and is used purely by the compiler.
- TypeScript assumes that you, the programmer, have performed any special checks that you need.
- Type assertions have two forms.
- One is the "as-syntax" and the other version is the "angle-bracket" syntax:

```
let someValue: any = "this is a string";  
let strLength: number = (someValue as string).length;
```

```
let someValue: any = "this is a string";  
let strLength: number = (<string>someValue).length;
```





Type Inference

- In TypeScript, there are several places where type inference is used to provide type information when there is no explicit type annotation. For example, in this code

```
let x = 3;  
// ^ = let x: number
```

- The type of the x variable is inferred to be number. This kind of inference takes place when initializing variables and members, setting parameter default values, and determining function return types.

```
let x; //Here x is inferred as 'any'  
x=10;  
x='Ram';  
x=true;
```

```
let x =10; //type inferred as number  
x=10;  
x='Ram'; //error  
x=true; //error
```

```
let x: number  
Type '"Ram"' is not assignable to type 'number'. ts(2322)  
Peek Problem (Alt+F8) No quick fixes available
```





Multiple Type

- When a type inference is made from several expressions, the types of those expressions are used to calculate a "best common type". For example:

```
let multitype : number | boolean;  
multitype=20;  
multitype=true;  
multitype='Ram'; //error
```

```
let multitype: number | boolean  
Type '"Ram"' is not assignable to type 'number | boolean'. ts(2322)  
Peek Problem (Alt+F8) No quick fixes available  
multitype='Ram'; //error
```

- Case Sensitivity : It can be tempting to think that the types Number, String, Boolean, Symbol, or Object are the same as the lowercase versions.
- These types do not refer to the language primitives however, and almost never should be used as a type. Instead, use the types number, string, boolean, object and symbol.

```
export {}  
let s:String='Ram Kumar';  
let f:string[];  
f=s.split(' ',1);  
console.log(f);
```





Functions

- Functions are the fundamental building block of any application in JavaScript.
- They're how you build up layers of abstraction, mimicking classes, information hiding, and modules.
- TypeScript also adds some new capabilities to the standard JavaScript functions to make them easier to work with.
- TypeScript functions can be created both as a named function or as an anonymous function.
- To quickly recap what these two approaches look like in JavaScript:

```
console.log('Named function-->' + add(10,20));  
// Named function  
function add(x, y) {  
    return x + y;  
}
```

```
// Anonymous function  
let myAdd = function (x, y) {  
    return x + y;  
};  
console.log('Anonymous function->' + myAdd(50,30));
```

Output →

```
PS D:\2020\Angular\TYPESCRIPT> node main.js  
Named function-->30  
Anonymous function->80  
PS D:\2020\Angular\TYPESCRIPT> █
```





Function – Parameters and Types

- We can add types to each of the parameters and then to the function itself to add a return type.
- TypeScript can figure the return type out by looking at the return statements, so we can also optionally leave this off in many cases.
- In JavaScript, every parameter is optional, and users may leave them off as they see fit.
- When they do, their value is undefined. We can get this functionality in TypeScript by adding a `?` to the end of parameters we want to be optional.
- In TypeScript, we can also set a value that a parameter will be assigned if the user does not provide one, or if the user passes undefined in its place.

```
export {}  
function add(x: number, y: number): number {  
    return x + y;  
}  
  
let myAdd = function (x: number, y: number): number {  
    return x + y;  
};
```

```
export {}  
function add(x: number, y?: number): number {  
    if(y)  
        return x + y;  
    else  
        return x;  
}  
  
console.log(add(5,10));  
console.log(add(25));
```

```
export {}  
function add(x: number,  
y: number=10): number {  
    return x + y;  
}  
  
console.log(add(5));  
console.log(add(12,20));
```





Datatype - Object

- object is a type that represents the non-primitive type, i.e. anything that is not number, string, boolean, symbol, null, or undefined.
- In the example below the function fullname has a parameter named person of type of an object has two string properties : firstname and lastname.

```
export {}  
function fullname(person: {firstname:string,lastname:string}){  
    console.log(`${person.firstname} ${person.lastname}`);  
}  
  
let p = {  
    firstname:'Ram',  
    lastname:'Kumar'  
}  
  
fullname(p);
```

- Here p is initialized object with 'Ram' and 'Kumar' as values to properties.
- Next invoking function.

```
PS D:\2020\Angular\TYPESCRIPT> node main.js  
Ram Kumar  
PS D:\2020\Angular\TYPESCRIPT> █
```

Output ↗





Interfaces

- In TypeScript, interfaces fill the role of naming these types, and are a powerful way of defining contracts within your code as well as contracts with code outside of your project.
- Basically it helps in handling many properties in an object.
- Let's do some changes in previous example, that will show uses.
- Interface can be used as a type that can be used in multiple functions.
- Even you can define a property optional in an interface as optional parameters used in functions using ? with property.

```
export {}  
interface Person{  
  firstname:string;  
  lastname:string;  
}  
function fullname(person:Person){  
  console.log(`${person.firstname} ${person.lastname}`);  
}
```

```
let p = {  
  firstname:'Ram',  
  lastname:'Kumar'  
}
```

```
fullname(p);
```

```
interface Person{  
  firstname:string;  
  lastname?:string;  
}
```

```
...  
let p = {  
  firstname:'Ram',  
}
```





Class & Objects

- Traditional JavaScript there is no concept of classes.
- They use functions and prototype-based inheritance to build up reusable components.
- This may feel a bit awkward to programmers more comfortable with an object-oriented approach, where classes inherit functionality and objects are built from these classes.
- With ECMAScript 6, JavaScript programmers can build their applications using this object-oriented class-based approach.
- The syntax should look familiar if you've used C# or Java before.





First Class

- We declared a new class Employee.
- This class has three members:
 - a property called employeeName;
 - a constructor, and
 - a method sayhello().
- You'll notice that in the class when we refer to one of the members of the class we prepend this.
- This denotes that it's a member access.
- With the let, we construct an instance of the Employee class using new.
- This calls into the constructor we defined earlier, creating a new object with the Employee shape, and running the constructor to initialize it.

```
export {}  
class Employee{  
  employeeName:string;  
  
  constructor(name:string) {  
    this.employeeName=name;  
  }  
  sayhello(){  
    console.log(`Hello! ${this.employeeName}`);  
  }  
}  
let emp1 = new Employee('Ram');  
  
console.log(emp1.employeeName);  
emp1.sayhello();
```

Output →

```
PS D:\2020\Angular\TYPESCRIPT> node main.js  
Ram  
Hello! Ram  
PS D:\2020\Angular\TYPESCRIPT> 
```





Inheritance

- In TypeScript, we can use common object-oriented patterns.
- One of the most fundamental patterns in class-based programming is being able to extend existing classes to create new ones using inheritance.
- The example below shows the most basic inheritance feature: class inherit properties and methods from base class.
- Derived classes are often called subclasses, and base classes are often called superclasses.

Output →

```
PS D:\2020\Angular\TYPESCRIPT> node main.js
Ram
Hello! Ram
Hello! Hari
Manager assigning tasks
Hari
PS D:\2020\Angular\TYPESCRIPT> 
```

```
export {}
class Employee{
  employeeName:string;
  constructor(name:string) {
    this.employeeName=name;
  }
  sayhello(){
    console.log(`Hello! ${this.employeeName}`);
  }
}
class manager extends Employee
{
  constructor(managerName:string) {
    super(managerName);
  }
  assignWork(){
    console.log(`Manager assigning tasks`);
  }
}
let emp1 = new Employee('Ram');
console.log(emp1.employeeName);
emp1.sayhello();

let emp2 = new manager('Hari');
emp2.sayhello();
emp2.assignWork();
console.log(emp2.employeeName);
```





Access Modifiers

- Access Modifiers are the keywords that set the accessibility of property or method in a class.
- Popularly public, private, and protected modifiers are used.
- Public is default means by default each class member is implicitly public and can be freely accessed through out the program.
- If you want private or protected you need to explicitly use keyword. Even you can use public keyword also, but not required.
- Private member can be accessed in their defined class only.
- Protected key word let you access member in It's own class and the classes derived from it. But beyond that they are inaccessible.





Examples - Get/Set Accessors

- TypeScript supports getters/setters as a way of intercepting accesses to a member of an object.
- This gives you a way of having finer-grained control over how a member is accessed on each object.

```
export {}  
const fullNameMaxLength = 10;  
class Employee {  
  private _fullName: string;  
  get fullName(): string {  
    return this._fullName;  
  }  
  set fullName(newName: string) {  
    if (newName && newName.length > fullNameMaxLength) {  
      throw new Error("fullName has a max length of " + fullNameMaxLength);  
    }  
    this._fullName = newName;  
  }  
}  
let employee = new Employee();  
employee.fullName = "Ram Kumar";  
if (employee.fullName) {  
  console.log(employee.fullName);  
}
```





Static Properties

- We can also create static members of a class, those that are visible on the class itself rather than on the instances.
- Each instance accesses this value through prepending the name of the class.
- Similarly to prepending this. in front of instance accesses, here we prepend Grid. in front of static accesses.

```
export {}  
class Grid {  
  static origin = { x: 0, y: 0 };  
  
  calculateDistanceFromOrigin(point: { x: number; y: number }) {  
    let xDist = point.x - Grid.origin.x;  
    let yDist = point.y - Grid.origin.y;  
    return Math.sqrt(xDist * xDist + yDist * yDist) / this.scale;  
  }  
  
  constructor(public scale: number) {}  
}  
  
let grid1 = new Grid(1.0); // 1x scale  
let grid2 = new Grid(5.0); // 5x scale  
  
console.log(grid1.calculateDistanceFromOrigin({ x: 10, y: 10 }));  
console.log(grid2.calculateDistanceFromOrigin({ x: 10, y: 10 }));
```





Abstract Classes

- Abstract classes are base classes from which other classes may be derived. They may not be instantiated directly. Unlike an interface, an abstract class may contain implementation details for its members. The abstract keyword is used to define abstract classes as well as abstract methods within an abstract class.

```
export {}
abstract class Department {
  constructor(public name: string) {}
  printName(): void { console.log("Department name: " + this.name); }
  abstract printMeeting(): void;
}
class AccountingDepartment extends Department {
  constructor() { super("Accounting and Auditing"); }
  printMeeting(): void {
    console.log("The Accounting Department meets each Monday at 10am.");
  }
  generateReports(): void {
    console.log("Generating accounting reports...");
  }
}
```

```
let department: Department; // ok to create a reference to an abstract type
//department = new Department(); // error: cannot create an instance of an abstract class
//Cannot create an instance of an abstract class.
department = new AccountingDepartment();
// ok to create and assign a non-abstract subclass
department.printName();
department.printMeeting();
//department.generateReports();
//Property 'generateReports' does not exist on type 'Department'.
```



Using a class as an interface

- As we said in the previous section, a class declaration creates two things: a type representing instances of the class and a constructor function. Because classes create types, you can use them in the same places you would be able to use interfaces.

```
export {}  
class Point {  
  x: number;  
  y: number;  
}  
interface Point3d extends Point {  
  z: number;  
}  
let point3d: Point3d = { x: 1, y: 2, z: 3 };  
console.log(`${point3d.x} --- ${point3d.y} --- ${point3d.z}`);
```